

---

## *WEEK-07 REPORT*

---



*(Node.js & Express Fundamentals)*

*Submitted by: Mohsin Ali*

*Role: Full Stack Intern*

*Email: [mohsinalirana222@gmail.com](mailto:mohsinalirana222@gmail.com)*

*Submitted on: August 18, 2026*

---

# Daily Progress Report

Week 7\_Day2: Node.js Core & File System

|             |                                       |
|-------------|---------------------------------------|
| Program     | 3-Month MERN Stack Internship Program |
| Intern Name | Mohsin Ali                            |
| Week        | Week_7 (Day 2)                        |
| Date        | 11 August, 2026                       |

## Day 2: Node.js Core & File System

---

### 1.1 Topics Studied

- Understanding Node.js core modules and their purpose
- Working with the built-in fs (File System) module
- Reading local files asynchronously using fs.readFile()
- Understanding asynchronous file system operations in Node.js
- Reading and processing JSON data from a local file
- Converting JSON strings into JavaScript objects and arrays using JSON.parse()
- Manipulating JavaScript arrays and objects
- Adding new data dynamically to an existing JSON dataset
- Converting JavaScript objects and arrays back into JSON using JSON.stringify()
- Writing updated JSON data back to the file system using fs.writeFile()
- Handling errors during asynchronous file operations
- Understanding callbacks in asynchronous Node.js operations
- Managing local file-based data using Node.js

### 1.2 Task Description

Developed a simple **Node.js User Data Management Script** to demonstrate practical usage of the built-in **File System (fs) module** and asynchronous file handling. The application reads user data stored in a local JSON file, parses the JSON content into a JavaScript array, adds a new mock user, and writes the updated user data back to the file system.

---

The application uses the asynchronous `fs.readFile()` method to retrieve the contents of the local JSON file without blocking the execution of the Node.js application. The retrieved JSON string is converted into a JavaScript array using `JSON.parse()`, allowing the user data to be manipulated programmatically.

A new mock user is then added to the existing array using JavaScript array manipulation. The updated array is converted back into properly formatted JSON using `JSON.stringify()` and saved to the original JSON file through the asynchronous `fs.writeFile()` method. Error handling was also implemented to manage potential issues during file reading and writing operations.

### 1.3 Implementation Details

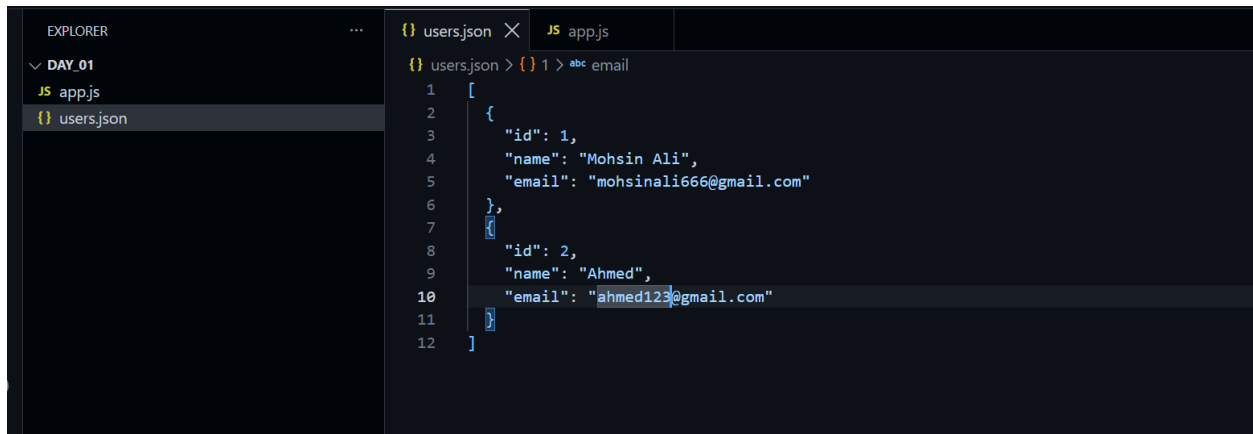
- Created a local `users.json` file containing sample user records.
- Created a Node.js script using the built-in `fs` module.
- Used `fs.readFile()` to asynchronously read user data from the JSON file.
- Used UTF-8 encoding to correctly read the JSON file as text.
- Implemented error handling for file reading operations.
- Used `JSON.parse()` to convert the JSON string into a JavaScript array.
- Created a new mock user object containing an ID, name, and email.
- Added the new user to the existing array using the `push()` method.
- Used `JSON.stringify()` to convert the updated JavaScript array back into JSON format.
- Used `fs.writeFile()` to asynchronously save the updated user data to the local JSON file.
- Implemented error handling for file writing operations.
- Used callback functions to handle the completion of asynchronous file system operations.
- Maintained readable JSON formatting using indentation during the conversion process.
- Verified that the new user was successfully added and persisted in the JSON file.

### 1.4 Outcome

Successfully developed a **Node.js User Data Management Script** demonstrating asynchronous file handling using the built-in **fs module**. The project strengthened the understanding of Node.js core modules, asynchronous programming, callback functions, JSON parsing and serialization, and basic file system operations.

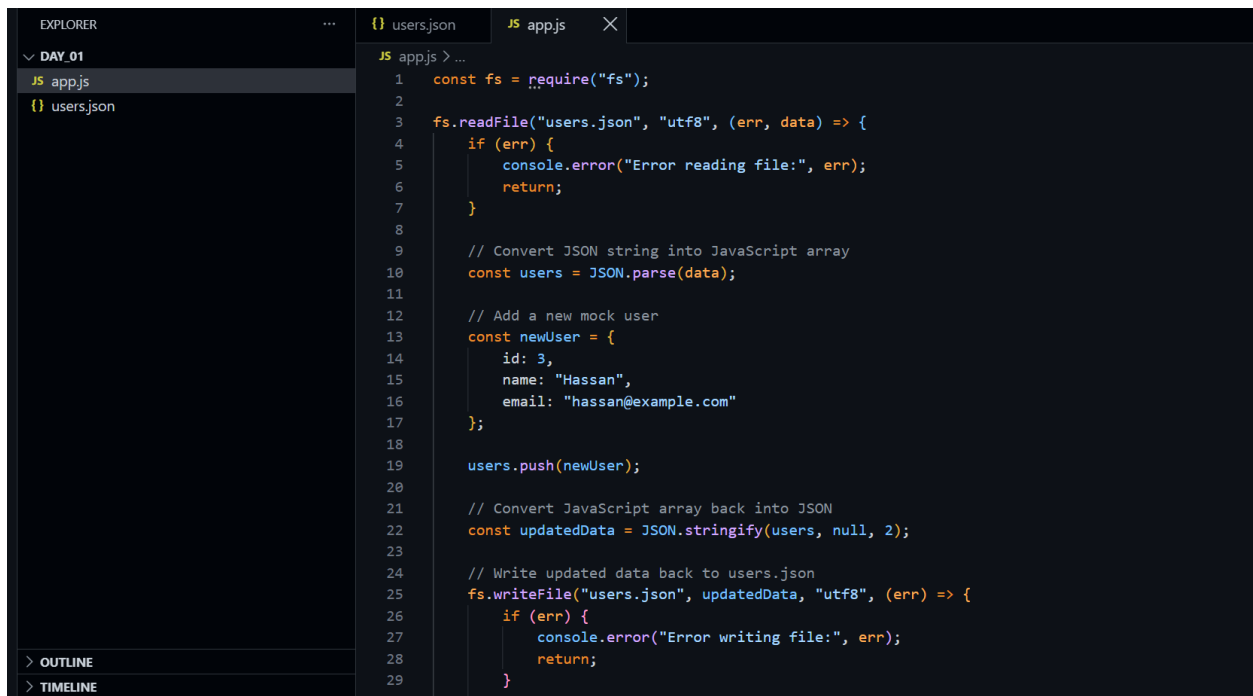
The implementation provided practical experience in reading, modifying, and writing local JSON data while maintaining proper error handling. This task also established a foundation for understanding how Node.js works with external data sources and file-based storage, which can later be extended to database-driven applications and backend APIs.

## Screenshots:



```
{} users.json X JS app.js

{} users.json > {} 1 > abc email
1  [
2    {
3      "id": 1,
4      "name": "Mohsin Ali",
5      "email": "mohsinali666@gmail.com"
6    },
7    {
8      "id": 2,
9      "name": "Ahmed",
10     "email": "ahmed123@gmail.com"
11   }
12 ]
```



```
EXPLORER ... {} users.json JS app.js X
v DAY_01
JS app.js
{} users.json

JS app.js > ...
1  const fs = require("fs");
2
3  fs.readFile("users.json", "utf8", (err, data) => {
4    if (err) {
5      console.error("Error reading file:", err);
6      return;
7    }
8
9    // Convert JSON string into JavaScript array
10   const users = JSON.parse(data);
11
12   // Add a new mock user
13   const newUser = {
14     id: 3,
15     name: "Hassan",
16     email: "hassan@example.com"
17   };
18
19   users.push(newUser);
20
21   // Convert JavaScript array back into JSON
22   const updatedData = JSON.stringify(users, null, 2);
23
24   // Write updated data back to users.json
25   fs.writeFile("users.json", updatedData, "utf8", (err) => {
26     if (err) {
27       console.error("Error writing file:", err);
28       return;
29     }
30   });
31 }
```

The screenshot shows a Visual Studio Code editor with two tabs: `users.json` and `app.js`. The `users.json` tab is active, displaying a JSON array of two user objects. The first object has an `id` of 2, a `name` of "Ahmed", and an `email` of "ahmed123@gmail.com". The second object has an `id` of 3, a `name` of "Hassan", and an `email` of "hassan@example.com". The Explorer sidebar on the left shows the file structure with `DAY_01` expanded, containing `app.js` and `users.json`. The Terminal panel at the bottom shows the command `node app.js` being executed, resulting in the output "New user added successfully!".

```
{} users.json X JS app.js
EXPLORER
└─ DAY_01
  └─ JS app.js
  {} users.json

{} users.json > {} 1 > abc email
6 },
7
8 "id": 2,
9 "name": "Ahmed",
10 "email": "ahmed123@gmail.com"
11 },
12 {
13 "id": 3,
14 "name": "Hassan",
15 "email": "hassan@example.com"
16 }
17 ]

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT
PS C:\Users\Mohsin Ali\Desktop\Internship\Glaxit\Week_07\Day_01> node app.js
New user added successfully!
PS C:\Users\Mohsin Ali\Desktop\Internship\Glaxit\Week_07\Day_01>
```

---

# Daily Progress Report

## Week 7\_Day2: Express.js Foundation & Routing

|             |                                       |
|-------------|---------------------------------------|
| Program     | 3-Month MERN Stack Internship Program |
| Intern Name | Mohsin Ali                            |
| Week        | Week_7 (Day 2)                        |
| Date        | 12 August, 2026                       |

## Day 2: Express.js Foundation & Routing

---

### 1.1 Topics Studied

- Understanding Express.js and its role in Node.js backend development
- Setting up an Express.js server and understanding the basic application structure
- Understanding the Request (req) and Response (res) objects
- Working with Express.js routing and HTTP methods
- Understanding GET, POST, PUT, and DELETE operations
- Understanding basic CRUD operations in a REST API
- Working with route parameters using req.params
- Handling JSON request data using req.body
- Using Express middleware such as express.json()
- Sending JSON responses using res.json()
- Understanding in-memory data storage using JavaScript arrays
- Understanding the limitations of testing APIs directly through a browser without a client UI

### 1.2 Task Description

Developed a simple **Express.js Task Management API** to demonstrate practical usage of Express.js for creating backend routes and handling basic CRUD operations. The application was initialized as a Node.js project and configured with Express.js to create a local development server.

The application uses an in-memory JavaScript array containing sample Task objects as temporary data storage. Different Express routes were created to retrieve, create, update, and delete tasks using the standard HTTP methods GET, POST, PUT, and DELETE.

---

The GET endpoints were tested directly through the web browser to retrieve task information. The limitations of browser-based testing were also explored, particularly the inability to conveniently send POST, PUT, and DELETE requests directly through the browser address bar without using a client interface or API testing tool.

### 1.3 Implementation Details

- Initialized a Node.js project using npm init
- Installed the Express.js package using npm
- Created a server.js file containing the Express application
- Created an Express application using express()
- Started the local server using app.listen() on port 3000
- Created an in-memory array containing sample Task objects
- Implemented a GET route to retrieve all available tasks
- Implemented a GET route using /:id to retrieve a specific task
- Implemented a POST route to create and add a new task
- Implemented a PUT route to update an existing task
- Implemented a DELETE route to remove a task from the array
- Used req.params to retrieve task IDs from route parameters
- Used req.body to access JSON data sent with POST and PUT requests
- Used express.json() middleware to process JSON request bodies
- Used res.json() to return task data and API responses
- Added basic error handling when a requested task could not be found
- Tested GET endpoints directly through the browser
- Observed that POST, PUT, and DELETE requests cannot be conveniently performed through the browser address bar
- Kept the task data in memory without connecting the application to a database

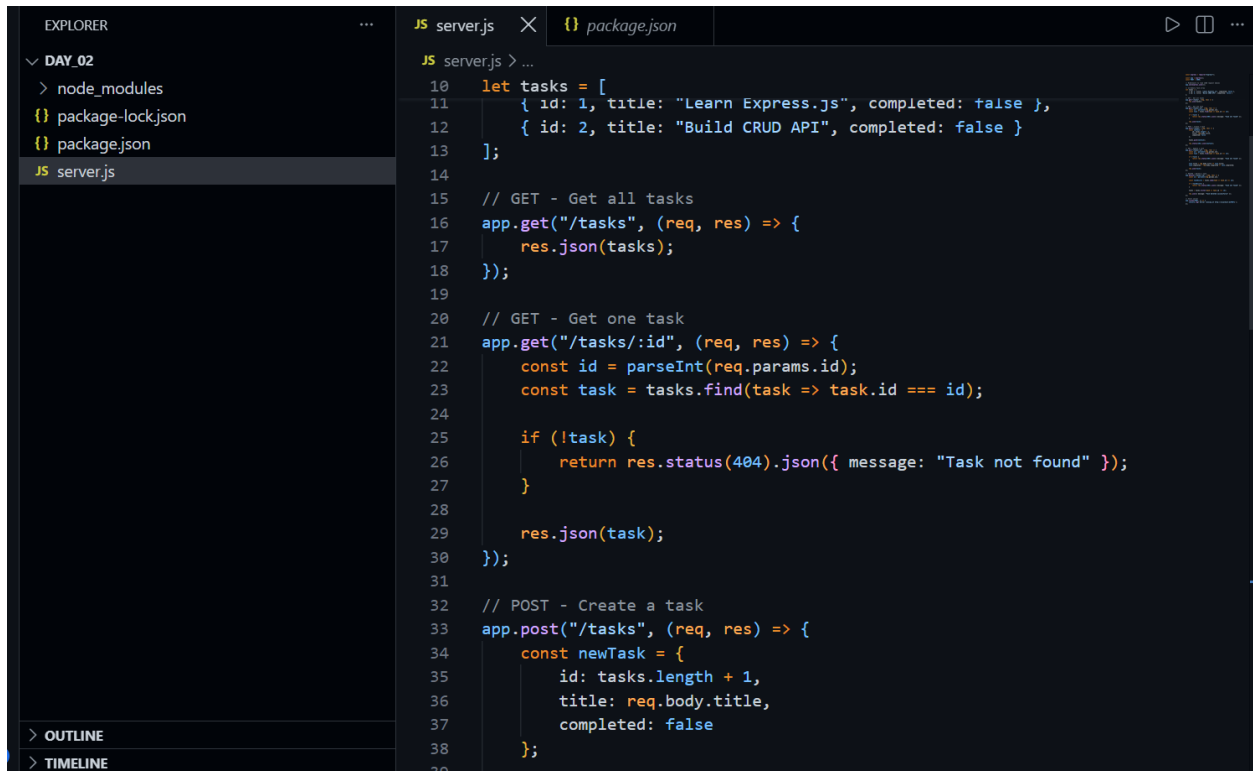
### 1.4 Outcome

Successfully developed a basic **Express.js Task Management API** demonstrating the fundamental concepts of Express server setup, routing, HTTP methods, request and response handling, middleware, and CRUD operations.

The implementation provided practical experience in creating REST API endpoints and working with temporary in-memory data. GET endpoints were successfully tested directly through the browser, while the limitations of browser-based API testing without a client UI were understood.

This task established a foundation for developing backend applications with Express.js and can later be extended by connecting the API to a database and integrating it with a frontend application.

## Screenshots:



The screenshot shows a VS Code editor with two files open: `server.js` and `package.json`. The `server.js` file contains the following code:

```
10 let tasks = [
11   { id: 1, title: "Learn Express.js", completed: false },
12   { id: 2, title: "Build CRUD API", completed: false }
13 ];
14
15 // GET - Get all tasks
16 app.get("/tasks", (req, res) => {
17   res.json(tasks);
18 });
19
20 // GET - Get one task
21 app.get("/tasks/:id", (req, res) => {
22   const id = parseInt(req.params.id);
23   const task = tasks.find(task => task.id === id);
24
25   if (!task) {
26     return res.status(404).json({ message: "Task not found" });
27   }
28
29   res.json(task);
30 });
31
32 // POST - Create a task
33 app.post("/tasks", (req, res) => {
34   const newTask = {
35     id: tasks.length + 1,
36     title: req.body.title,
37     completed: false
38   };
39 }
```

← → ↻ ⓘ localhost:3000/tasks

Pretty print ☒

```
[
  {
    "id": 1,
    "title": "Learn Express.js",
    "completed": false
  },
  {
    "id": 2,
    "title": "Build CRUD API",
    "completed": false
  }
]
```

---

# Daily Progress Report

## Week 7\_Day3: Middlewares & Data Parsing

|             |                                       |
|-------------|---------------------------------------|
| Program     | 3-Month MERN Stack Internship Program |
| Intern Name | Mohsin Ali                            |
| Week        | Week_7 (Day 3)                        |
| Date        | 13 August, 2026                       |

## Day 3: Middlewares & Data Parsing

---

### 1.1 Topics Studied

- Understanding the middleware pipeline in Express.js
- Understanding how middleware processes incoming HTTP requests
- Working with the built-in `express.json()` middleware
- Understanding JSON request body parsing using `req.body`
- Working with third-party middleware in Express.js
- Using `morgan` for HTTP request logging
- Using `cors` middleware for Cross-Origin Resource Sharing
- Creating and using custom Express middleware
- Understanding the purpose of the `next()` function
- Understanding middleware execution order
- Logging HTTP methods, URLs, and timestamps for incoming requests

### 1.2 Task Description

Developed a simple **Express.js Middleware and Data Parsing application** to demonstrate the practical use of Express middleware. The application uses the built-in `express.json()` middleware to parse JSON data received through POST requests.

A custom logging middleware was also implemented to record the HTTP method, requested URL, and timestamp of every incoming request. Third-party middleware including `morgan` and `cors` was also integrated to understand how external middleware can be used in an Express.js application.

---

The application includes basic GET and POST routes to test middleware execution and JSON request body parsing.

### 1.3 Implementation Details

- Created an Express.js application using `express()`
- Installed and configured the `cors` and `morgan` middleware packages
- Used `express.json()` to parse JSON request bodies
- Created a custom logging middleware in a separate `logger.js` file
- Used `req.method` to retrieve the HTTP method of incoming requests
- Used `req.originalUrl` to retrieve the requested URL
- Used `new Date().toISOString()` to generate the request timestamp
- Used `next()` to pass requests to the next middleware or route
- Added the custom logger to the Express middleware pipeline
- Used `morgan("dev")` as a third-party request logging middleware
- Used `cors()` as third-party middleware
- Created a GET route to test incoming requests
- Created a POST route to receive JSON user data
- Used `req.body` to access parsed JSON data
- Used `res.json()` to return JSON responses
- Tested GET and POST requests using the browser and API testing tools
- Observed custom middleware logs in the server terminal

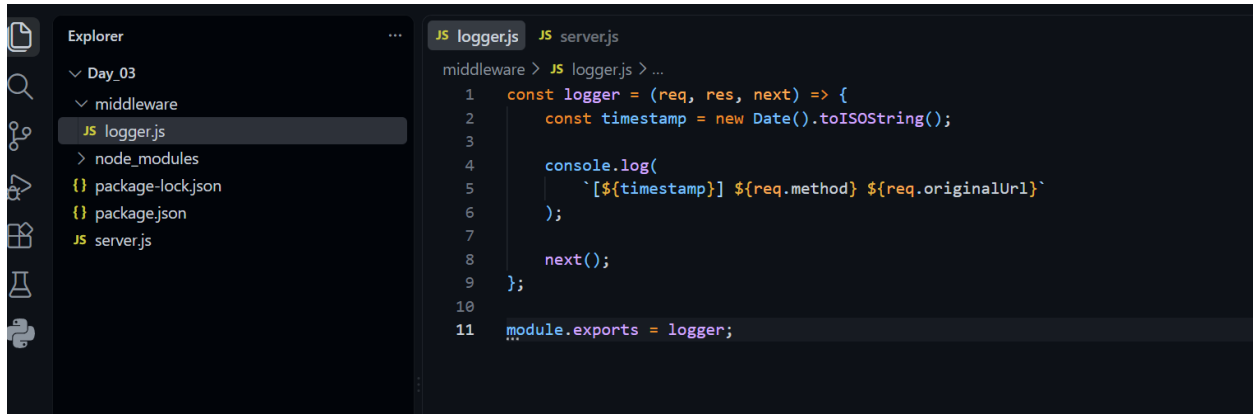
### 1.4 Outcome

Successfully implemented an **Express.js Middleware and Data Parsing application** demonstrating the middleware pipeline, built-in and third-party middleware, custom middleware, and JSON body parsing.

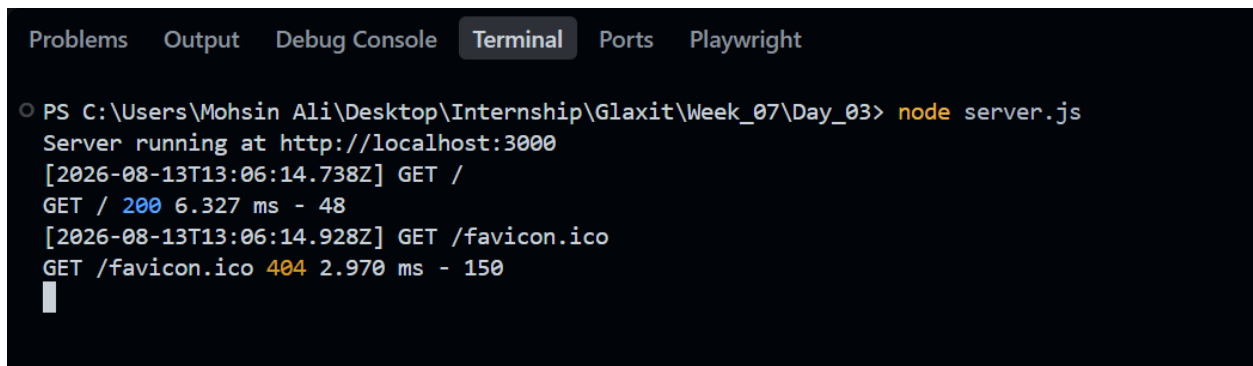
The custom logging middleware successfully recorded the HTTP method, URL, and timestamp of incoming requests. The application also successfully processed JSON data using `express.json()` and accessed the parsed information through `req.body`.

This task provided practical understanding of how middleware works in Express.js and how it can be used for request processing, logging, and other common backend functionality.

## Screenshots:



```
JS logger.js JS server.js
middleware > JS logger.js > ...
1  const logger = (req, res, next) => {
2      const timestamp = new Date().toISOString();
3
4      console.log(
5          `[${timestamp}] ${req.method} ${req.originalUrl}`
6      );
7
8      next();
9  };
10
11  module.exports = logger;
```



```
Problems Output Debug Console Terminal Ports Playwright
PS C:\Users\Mohsin Ali\Desktop\Internship\Glaxit\Week_07\Day_03> node server.js
Server running at http://localhost:3000
[2026-08-13T13:06:14.738Z] GET /
GET / 200 6.327 ms - 48
[2026-08-13T13:06:14.928Z] GET /favicon.ico
GET /favicon.ico 404 2.970 ms - 150
```

Pretty print ☐

```
{"message":"Day 3 Middleware Server is Running"}
```

---

# Daily Progress Report

## Week 7\_Day4: REST Principles & Postman

|             |                                       |
|-------------|---------------------------------------|
| Program     | 3-Month MERN Stack Internship Program |
| Intern Name | Mohsin Ali                            |
| Week        | Week_7 (Day 4)                        |
| Date        | 15 August, 2026                       |

## Day 4: REST Principles & Postman

---

### 1.1 Topics Studied

- RESTful API principles and resource-based URI design using plural resource names and standard HTTP methods.
- HTTP status codes such as 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found, and 500 Internal Server Error.
- Testing and validating backend API endpoints using Postman with JSON request bodies.
- Request validation and handling malformed JSON requests in Express.js.

### 1.2 Task Description

The task was to refine an Express.js API according to REST principles and implement a complete CRUD workflow for task management. Standard HTTP methods (GET, POST, PUT, and DELETE) were used for different operations. The API was configured to process JSON requests, validate incoming data, return appropriate HTTP status codes, and provide relevant response headers. All implemented endpoints were tested using Postman

### 1.3 Implementation Details

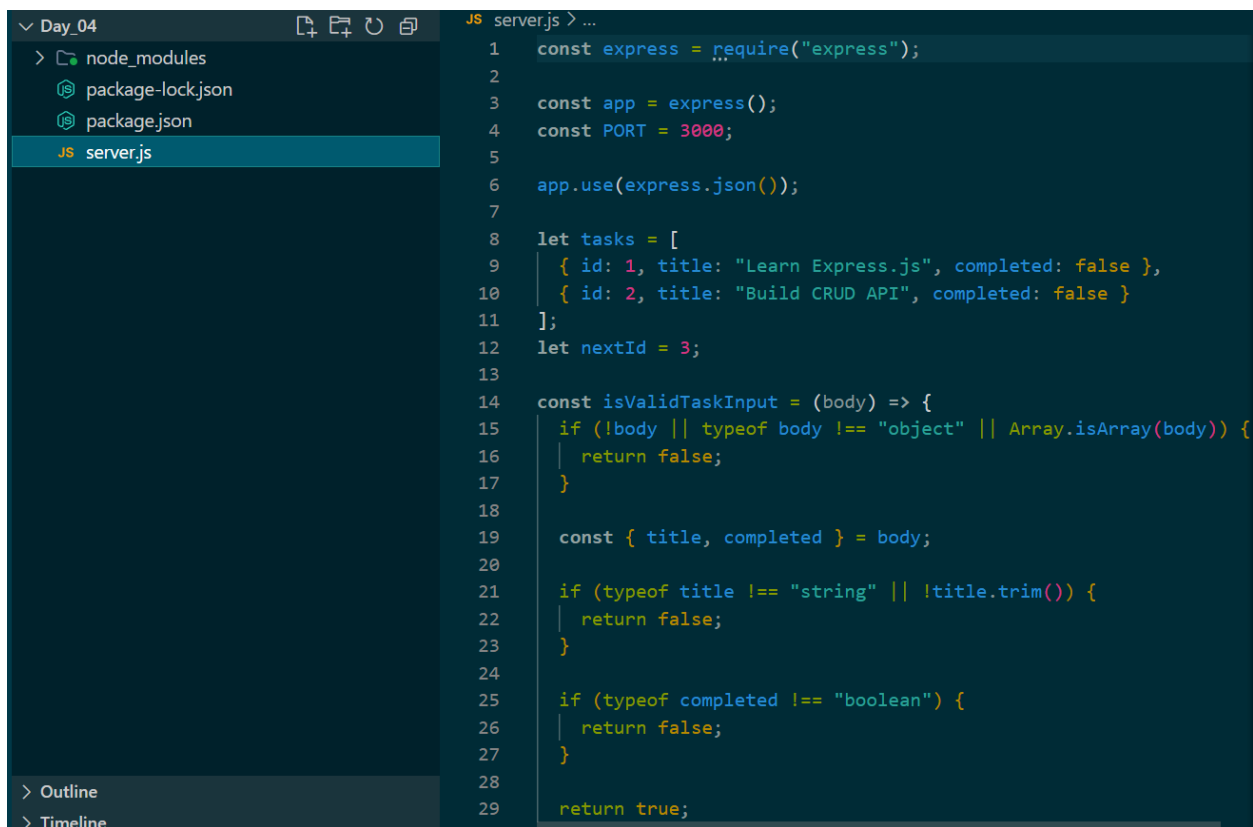
- Configured `express.json()` middleware to process incoming JSON request bodies.
- Structured API routes using REST conventions with `/tasks` and `/tasks/:id`.
- Implemented input validation to ensure that title is a non-empty string and completed is a boolean.
- Used appropriate HTTP status codes for successful requests and different error conditions.

- Used 201 Created with a Location header when a new task was successfully created.
- Added error-handling middleware to handle malformed JSON requests and return a proper 400 Bad Request response.
- Tested the complete CRUD functionality through Postman using different request methods, JSON payloads, and response headers.

## 1.4 Outcome

Successfully developed and tested a RESTful task management API using Express.js. The API performs CRUD operations, validates incoming data, handles invalid and malformed requests, and returns appropriate HTTP responses. All endpoints were tested through Postman to verify their functionality and expected behavior.

### Screenshots:



```
JS server.js > ...
1  const express = require("express");
2
3  const app = express();
4  const PORT = 3000;
5
6  app.use(express.json());
7
8  let tasks = [
9    { id: 1, title: "Learn Express.js", completed: false },
10   { id: 2, title: "Build CRUD API", completed: false }
11 ];
12 let nextId = 3;
13
14 const isValidTaskInput = (body) => {
15   if (!body || typeof body !== "object" || Array.isArray(body)) {
16     return false;
17   }
18
19   const { title, completed } = body;
20
21   if (typeof title !== "string" || !title.trim()) {
22     return false;
23   }
24
25   if (typeof completed !== "boolean") {
26     return false;
27   }
28
29   return true;
30 }
```

---

```
[
  {
    "id": 1,
    "title": "Learn Express.js",
    "completed": false
  },
  {
    "id": 2,
    "title": "Build CRUD API",
    "completed": false
  },
  {
    "id": 3,
    "title": "Test Task from Postman Alternative",
    "completed": false
  }
]
```

---

---

---

---

# Daily Progress Report

## Week 7\_Day5: Mini-Project - Static JSON API

|             |                                       |
|-------------|---------------------------------------|
| Program     | 3-Month MERN Stack Internship Program |
| Intern Name | Mohsin Ali                            |
| Week        | Week_7 (Day 5)                        |
| Date        | 17 August, 2026                       |

## Day 5: Mini-Project - Static JSON API

---

### 1.1 Topics Studied

- Building a full-stack static JSON RESTful API with Node.js and Express.js.
- Asynchronous file system persistence using fs.promises (readFile and writeFile).
- Implementing full CRUD operations with dynamic route parameters (/api/books/:id).
- Enforcing input payload validation, query parameter filtering, and strict HTTP status code responses (200, 201, 204, 400, 404, 500).

### 1.2 Task Description

Developed a complete RESTful Bookstore API mini-project that reads and writes resource data to a local books.json file. The application provides complete CRUD functionality, allowing users to list books, search/filter by genre or author, retrieve individual books by ID, create new books with auto-incrementing IDs, update existing book details, and delete specified items. All state changes are asynchronously persisted to the local JSON file.

### 1.3 Implementation Details

- Created a structured Day\_05 project folder containing server.js, package.json, and a data/books.json database file.
- Configured Express middleware express.json() and built helper functions (readBooks and writeBooks) using fs.promises.
- Implemented route parameter handling for /api/books/:id across GET, PUT, and DELETE requests.
- Built input validation logic (validateBookPayload) checking required fields (title, author, genre, price, publishedYear) and data types.

- Configured standard REST status codes (201 Created with Location header on insertion, 204 No Content on deletion, 400 Bad Request on invalid payload/ID, and 404 Not Found on missing resources).
- Added query string filtering capabilities (?genre=, ?author=, ?search=) on the GET /api/books route.
- Executed automated end-to-end testing to verify HTTP status codes and confirm persistent file updates in data/books.json.

## 1.4 Outcome

Successfully delivered a fully functional static JSON Bookstore API with complete CRUD capabilities, route parameter support, input validation, and local file system persistence. All operations return accurate HTTP status codes and structured JSON outputs, completing the Day 5 mini-project requirements.

### Screenshots:

```
13
14  const isValidTaskInput = (body) => {
15    if (!body || typeof body !== "object" || Array.isArray(body)) {
16      return false;
17    }
18
19    const { title, completed } = body;
20
21    if (typeof title !== "string" || !title.trim()) {
22      return false;
23    }
24
25    if (typeof completed !== "boolean") {
26      return false;
27    }
28    Ctrl+L for Agent
29    return true;
30  };
31
32  const taskNotFoundResponse = (res, id) => {
33    return res.status(404).json({
34      message: `Task with id ${id} was not found.`
35    });
36  };
37
38  app.get("/", (req, res) => {
39    res.status(200).json({
40      message: "Task API is running. Use /tasks to view all tasks."
41    });
42  });
```

---

```
{
  "count": 4,
  "data": [
    {
      "id": 1,
      "title": "To Kill a Mockingbird (Special Edition)",
      "author": "Harper Lee",
      "genre": "Classic Fiction",
      "price": 19.99,
      "publishedYear": 1960
    },
    {
      "id": 3,
      "title": "The Great Gatsby",
      "author": "F. Scott Fitzgerald",
      "genre": "Classic",
      "price": 10.99,
      "publishedYear": 1925
    },
    {
      "id": 4,
      "title": "The Hobbit",
      "author": "J.R.R. Tolkien",
      "genre": "Fantasy",
      "price": 15.99,
      "publishedYear": 1937
    },
    {
      "id": 5,
      "title": "Clean Code",
      "author": "Robert C. Martin",
      "genre": "Software Engineering",
      "price": 39.99,
      "publishedYear": 2008
    }
  ]
}
```

---

---